

**Practical File
Of
Operating Systems Laboratory
(UGCA1926)**



Submitted By: -
Bikkar Singh
Roll No. 2447072
Class: BCA 4th Sem

Submitted to: -
Ms. Baljit Kaur
Assistant professor
(Dept. of Computer Application)

**PUNJAB INSTITUTE OF MANAGEMENT AND TECHNOLOGY
(AFFILIATED TO IKG PUNJAB TECHNICAL UNIVERSITY, JALANDHAR)**

Sr No.	Practical	Page No.	Signature
1	Installation of Windows OS	3	
2	Installation of Linux OS	5	
3	Dual Boot Installation of Operating Systems	7	
4	Implementation of FCFS Scheduling Algorithm	9	
5	Implementation of SJF Scheduling Algorithm	12	
6	Implementation of Round-Robin Scheduling Algorithm	14	
7	Vi Editor and Its Commands	16	
8	Shell Commands	18	
9	Shell Scripting - Using Variables	20	
10	Shell Scripting - Input & Output	22	
11	Shell Scripting - Data Types	24	
12	Shell Scripting - Use of Arithmetic Operators	25	
13	Shell Scripting - If Control Statement Programs	27	
14	Shell Scripting - While Control Statement	29	
15	Shell Scripting - For Control Statement	30	

Practical No. 1: Installation of Windows OS

Aim:

To install Windows Operating System on a computer system.

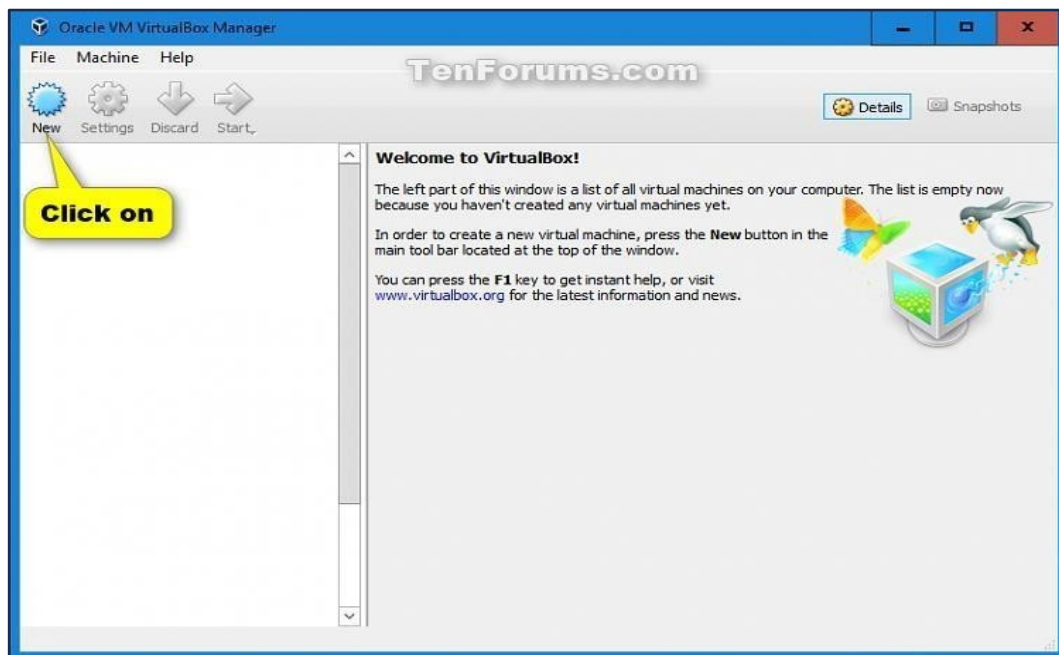
Requirements:

- Bootable USB or DVD with Windows ISO
- Computer with minimum system requirements
- Keyboard and mouse
- Stable power supply

Procedure:

1. Insert the bootable Windows USB/DVD into the system.
2. Restart the system and enter the BIOS/Boot menu by pressing keys like F2, F12, DEL, or ESC (depends on manufacturer).
3. From the boot menu, select the USB/DVD to boot.
4. Wait for the Windows setup screen to appear.
5. Choose the language, time zone, and keyboard input method, then click **Next**.
6. Click on **Install Now**.
7. Enter the product key if available, or skip it.
8. Select the edition of Windows you want to install.
9. Accept the license agreement.
10. Choose **Custom: Install Windows only (advanced)**.
11. Select the drive where you want to install Windows (format if needed).
12. Click **Next** and wait for the installation to complete.
13. After restarting, complete initial setup: region, account name, password, privacy settings.

Output:



Practical No. 2: Installation of Linux OS

Aim:

To install a Linux distribution (e.g., Ubuntu) on a computer system.

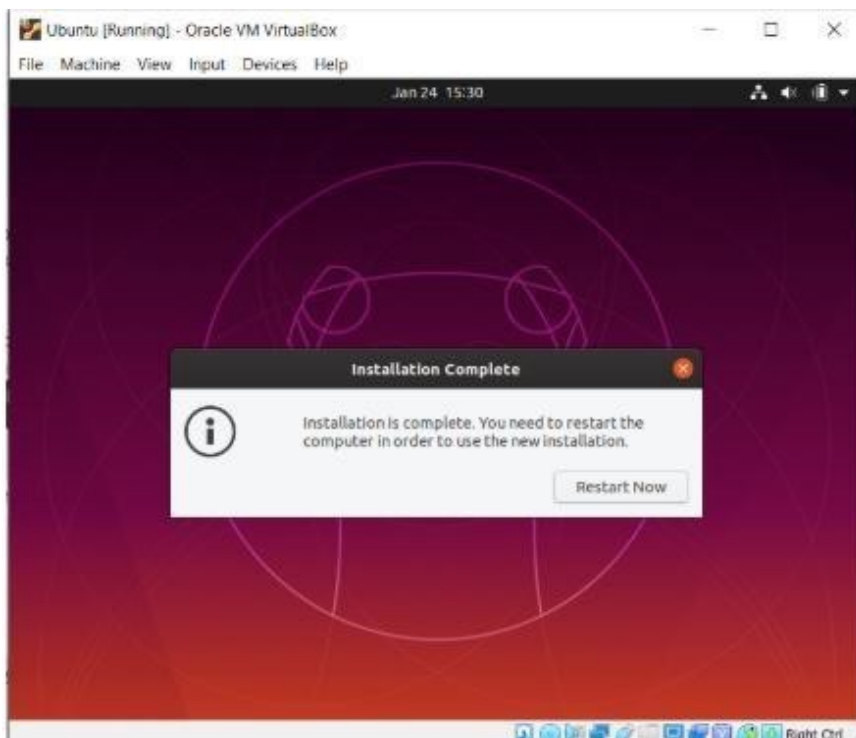
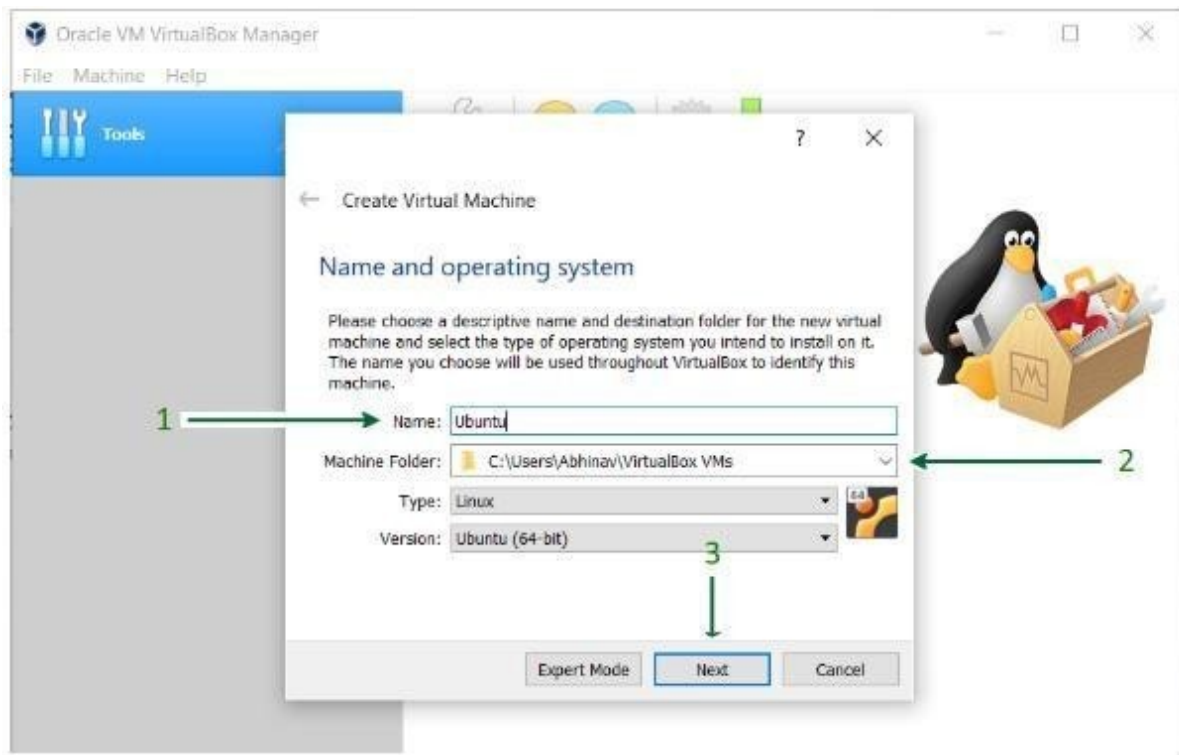
Requirements:

- Bootable USB drive with Linux ISO (Ubuntu, Fedora, etc.)
- Rufus software to create bootable USB
- Computer with at least 4GB RAM
- Internet connection (optional)

Procedure:

1. Download the ISO file of the Linux distribution (e.g., ubuntu-22.04.iso).
2. Use **Rufus** to create a bootable USB with the ISO file.
3. Insert the USB and restart the system.
4. Enter the BIOS/boot menu and select the USB device to boot.
5. On the welcome screen, select “Install Ubuntu”.
6. Choose the keyboard layout and click **Continue**.
7. Select installation type: **Erase disk** (fresh install) or **Install alongside Windows** (dual boot).
8. Configure partitions if needed, or let installer handle it.
9. Enter your name, computer name, username, and password.
10. Click **Install Now** and confirm changes.
11. Wait for the installation to complete, then restart the computer.
12. Remove the USB when prompted.

Output:



Practical No. 3: Dual Boot Installation of Operating Systems

Aim:

To install and configure a dual-boot system with Windows and Linux operating systems.

Requirements:

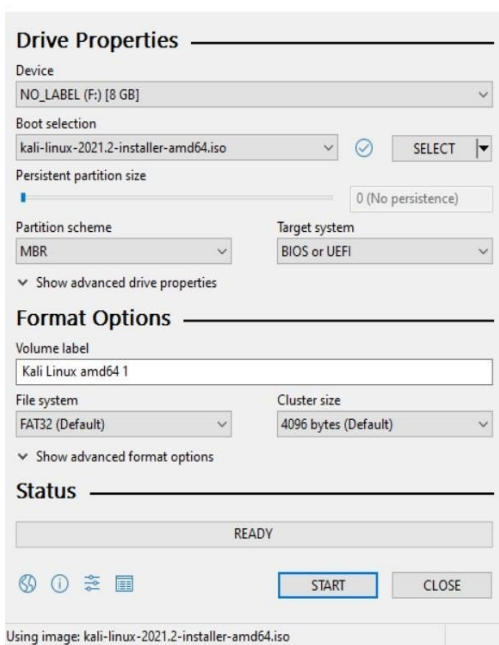
- Bootable USBs for both Windows and Linux
- Computer with minimum 60 GB free space
- Rufus tool (for USB bootable)

Procedure:

1. First install **Windows OS** normally (if not already installed).
2. Boot into Windows and shrink disk space (around 30–50 GB) using **Disk Management**.
3. Insert the **bootable Linux USB** and restart the system.
4. Enter BIOS/boot menu and boot from USB.
5. On Linux setup screen, select **Install Ubuntu**.
6. Choose **Install Ubuntu alongside Windows Boot Manager** (automatic dual boot).
 - Or choose **Something else** for manual partitioning:
 - Select the free space
 - Create / root partition (min 25 GB)
 - Create **swap** (optional, ~2 GB)
 - Create /home (optional)
7. Select the device for bootloader installation (usually /dev/sda).
8. Continue installation and complete user setup.
9. Restart system after installation.

10. On boot, you will see the **GRUB boot menu** to choose between Windows and Linux.

Output:



Practical No. 4: FCFS (First Come First Serve) Scheduling Algorithm

Aim:

To implement and understand the First Come First Serve (FCFS) scheduling algorithm.

Requirements:

- Programming environment (C/Python)
- Input: Process IDs and burst times

Procedure (Example in C++):

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    // Predefined number of processes and their burst times
```

```
    int n = 4;
```

```
    int bt[] = {5, 8, 12, 6}; // Burst time for P1, P2, P3, P4
```

```
    int wt[n], tat[n];
```

```
    // First process waiting time is always 0
```

```
    wt[0] = 0;
```

```
    // Calculate waiting time for each process
```

```
    for (int i = 1; i < n; i++) {
```

```
        wt[i] = wt[i - 1] + bt[i - 1]; }
```

```
    // Calculate turnaround time for each process
```

```
    for (int i = 0; i < n; i++) {
```

```
        tat[i] = wt[i] + bt[i];
```

```
    }
```

```

// Display process details

cout <<

"Process\tBT\tWT\tTAT\n"; for (int

i = 0; i < n; i++) {

    cout << "P" << i + 1 << "\t" << bt[i] << "\t" << wt[i] << "\t" << tat[i] <<

endl;

}

return 0;

}

```

Output:

FCFS (First Come First Serve) – Example

► Step-by-step Calculation:

Process	BT	WT	TAT
P1	5	0	5
P2	8	5	13
P3	12	13	25

Formulas:

- Waiting Time (WT) = Start Time – Arrival Time
- Turnaround Time (TAT) = Completion Time – Arrival Time

Gantt Chart:

| P1 | P2 | P3 |

0 5 13 25

Average WT: $(0 + 5 + 13) / 3 = 6$

Average TAT: $(5 + 13 + 25) / 3 = 14.33$

Practical No. 5: SJF (Shortest Job First) Scheduling Algorithm

Aim:

To implement the Shortest Job First (SJF) scheduling algorithm and observe the turnaround and waiting times.

Requirements:

- Programming environment (C/Python)
- Input: Process IDs and burst times

Procedure (C++ Example):

```
#include <iostream>

using namespace std;

int main() {
    int n = 4;

    int pid[4] = {1, 2, 3, 4};    // Process IDs
    int bt[4] = {5, 8, 12, 6};    // Burst
    Times int wt[4], tat[4];

    // Sort processes based on burst time using basic logic
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++)
            { if (bt[i] > bt[j]) {
                // manually swapping burst time

                int temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                // also swap process IDs

                int temp_pid = pid[i];
```

```

        pid[i] = pid[j];

        pid[j] = temp_pid;} } }

// Calculate waiting time

wt[0] = 0;

for (int i = 1; i < n; i++) {

    wt[i] = wt[i - 1] + bt[i - 1];

}

// Calculate turnaround time

for (int i = 0; i < n; i++) {

    tat[i] = wt[i] + bt[i]; }

return 0;}

```

Output:

SJF (Shortest Job First) – Example

► Sort by Burst Time:

P2 (8), P1 (5), P3 (12)

Process	Burst Time (BT)	Waiting Time (WT)	Turnaround Time (TAT)
P1	5	0	5
P2	8	11	19
P3	12	19	31

Gantt Chart:

| P2 | P1 | P3 |

0 5 13 25

Average WT: $(0 + 11 + 19) / 3 = 10$

Average TAT: $(5 + 19 + 31) / 3 = 18.33$

Practical No. 6: Round Robin Scheduling Algorithm

Aim:

To implement the Round Robin scheduling algorithm with a given time quantum.

Requirements:

- Programming language (C/Python)
- Process list with burst times
- Time quantum value

Procedure (C++ Example):

```
#include <iostream>

using namespace std;

int main() {

    int n = 3, int pid[3] = {1, 2, 3}; // Process IDs, int bt[3] = {5, 4, 6}; // Burst
    Times

    int rt[3];                // Remaining Times int wt[3] = {0};
                              // Waiting Times

    int tat[3];              // Turnaround Times

    int tq = 2;              // Time
    Quantum for (int i = 0; i < n; i++) {

        rt[i] = bt[i]; }

    int time = 0;

    bool done;

    do {done = true;

        for (int i = 0; i < n; i++)

            { if (rt[i] > 0) {

                done = false;
```

```

    if (rt[i] > tq) {
        time += tq; rt[i] -= tq;
    } else {
        time += rt[i];
        wt[i] = time - bt[i];
        rt[i] = 0; } } }
} while (!done);
for (int i = 0; i < n; i++)
    { tat[i] = wt[i] + bt[i]; }
return 0;}

```

Output:

Round Robin Algorithm– Example

► Sort by Burst Time:

P2 (5), P1 (4), P3 (6)

Process	Burst Time (BT)	Waiting Time (WT)	Turnaround Time (TAT)
P1	5	8	13
P2	4	6	10
P3	6	9	15

Gantt Chart:

| P1 | P2 | P3 | P1 | P2 | P3 | P1 | P3 |

0 2 4 6 8 10 12 13 15

Average WT = $(8 + 6 + 9) / 3 = 7.67$

Average TAT = $(13 + 10 + 15) / 3 = 12.67$

Practical No. 7: Vi Editor Commands

Aim:

To learn basic file editing using the Vi editor in Linux.

Requirements:

- Linux terminal
- Vi installed (default in most Linux distros)

Procedure:

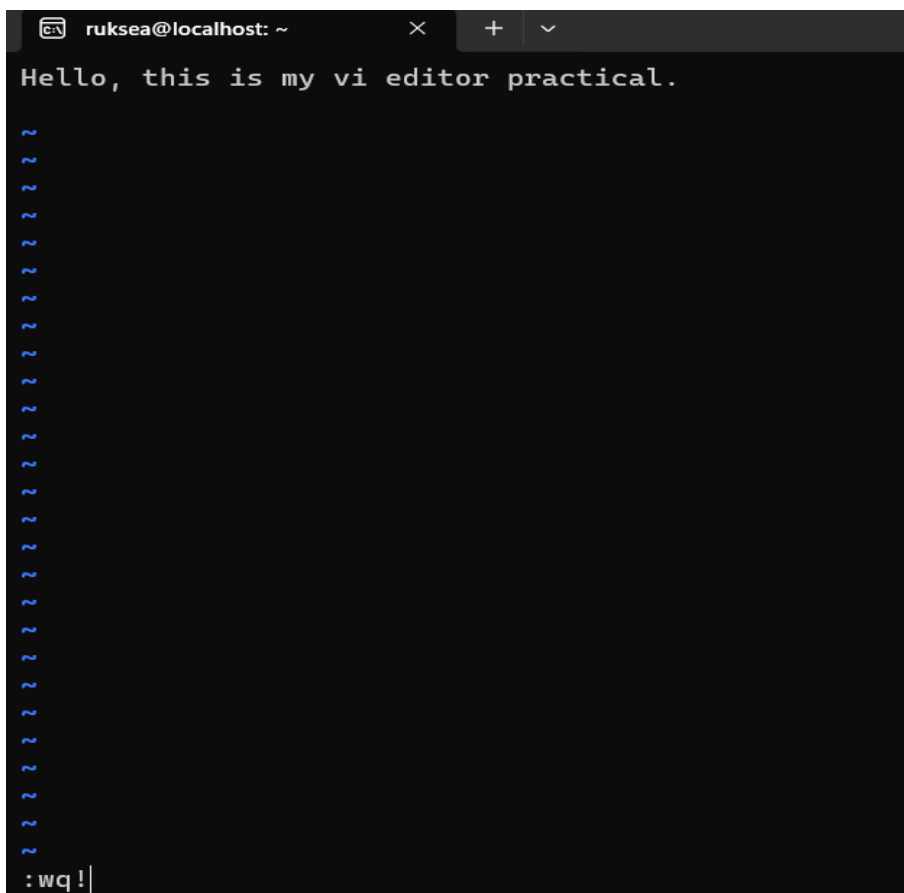
1. Open terminal.
2. To create/edit file: vi filename.txt
3. Press i to enter **insert mode**.
4. Type your content.
5. Press Esc to exit insert mode.

Basic Commands:

Command	Description
vi filename	Open or create a file in
vi I	Insert text before cursor
A	Insert text after cursor
O	Open a new line below cursor
X	Delete character under
cursor Dd	Delete current line
Yy	Copy current line
P	Paste copied line below
cursor U	Undo last change

Command	Description
:	Enter last line mode (e.g. :wq)
:w	Save the file
:q	Quit vi
:wq	Save and quit
:q!	Quit without saving

Output:



```

ruksea@localhost: ~
Hello, this is my vi editor practical.
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
:wq!

```

Practical No. 8: Shell Commands

Aim:

To use basic Linux shell commands for file and directory management.

Requirements:

- Linux shell (bash)
- Terminal access

Basic Shell Commands :

Command	Description
<code>pwd</code>	Show current working directory
<code>ls</code>	List files and folders
<code>cd foldername</code>	Change directory
<code>mkdir folder</code>	Create new folder
<code>rmdir folder</code>	Remove empty folder
<code>touch file.txt</code>	Create an empty file
<code>cat file.txt</code>	Show contents of a file
<code>cp file1 file2</code>	Copy file1 to file2
<code>mv old new</code>	Rename or move file/folder
<code>rm file.txt</code>	Delete a file
<code>clear</code>	Clear terminal screen
<code>exit</code>	Close the terminal

Output:

```
ruksea@localhost: ~/newfold × + v
(ruksea@localhost)-[~]
$ pwd
/home/ruksea
(ruksea@localhost)-[~]
$ mkdir newfolder
(ruksea@localhost)-[~]
$ ls
demo.txt  example.txt  myfolder  newfolder
(ruksea@localhost)-[~]
$ cd newfolder
(ruksea@localhost)-[~/newfolder]
$ touch demo1.txt
(ruksea@localhost)-[~/newfolder]
$ ls
demo1.txt
(ruksea@localhost)-[~/newfolder]
$ cat demo1.txt
(ruksea@localhost)-[~/newfolder]
$ rm demo1.txt
(ruksea@localhost)-[~/newfolder]
$ clear
```

Practical No. 9: Shell Script – Using Variables

Aim:

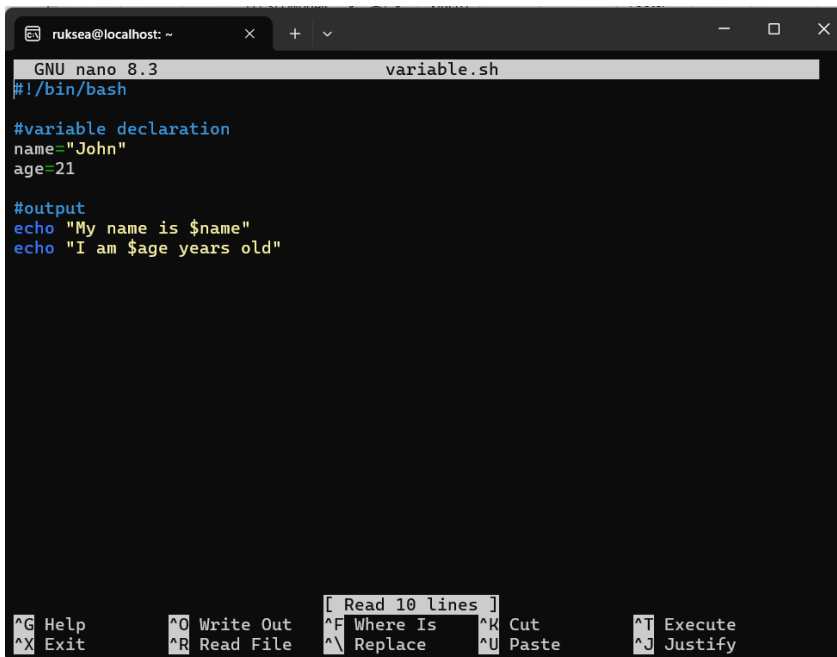
To create a shell script that uses variables.

Requirements:

- Linux environment
- bash shell

Procedure:

1. Open terminal and type: nano variable.sh
2. Write the script:



```
GNU nano 8.3 variable.sh
#!/bin/bash

#variable declaration
name="John"
age=21

#output
echo "My name is $name"
echo "I am $age years old"
```

3. Save the file and exit.
Press Ctrl + X → Y → Enter
4. Make it executable:

 chmod +x
 variable.sh
5. Run the script:

 ./variable.sh

Output:

```
ruksea@localhost: ~  
  
(ruksea@localhost)~  
$ nano variable.sh  
  
(ruksea@localhost)~  
$ chmod +x variable.sh  
  
(ruksea@localhost)~  
$ ./variable.sh  
My name is John  
I am 21 years old  
  
(ruksea@localhost)~  
$ ls  
demo.txt  example.txt  myfolder  newfolder  variable.sh  
  
(ruksea@localhost)~  
$ ls -l variable.sh  
-rwxr-xr-x 1 ruksea ruksea 115 Apr 10 13:59 variable.sh  
  
(ruksea@localhost)~  
$ |
```

Practical No. 10: Shell Script – Input/Output

Aim:

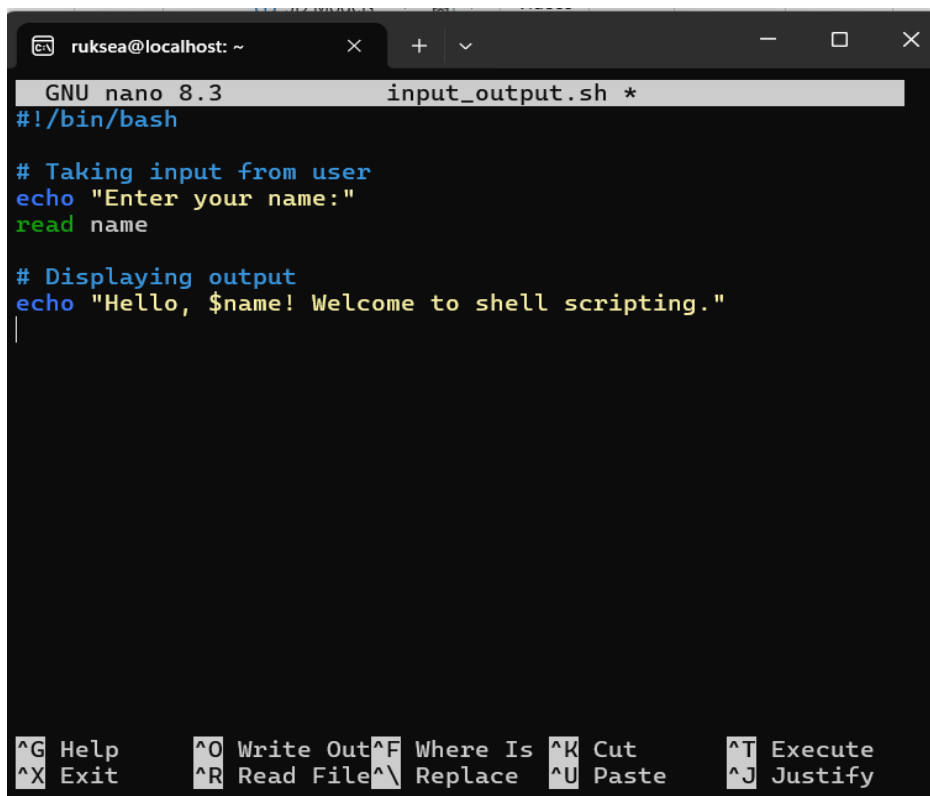
To create a shell script that takes input from user and displays output.

Requirements:

- Linux environment
- bash shell

Procedure:

1. Open terminal and type: nano input_output.sh
2. Write the script:



```
GNU nano 8.3 input_output.sh *
#!/bin/bash

# Taking input from user
echo "Enter your name:"
read name

# Displaying output
echo "Hello, $name! Welcome to shell scripting."
```

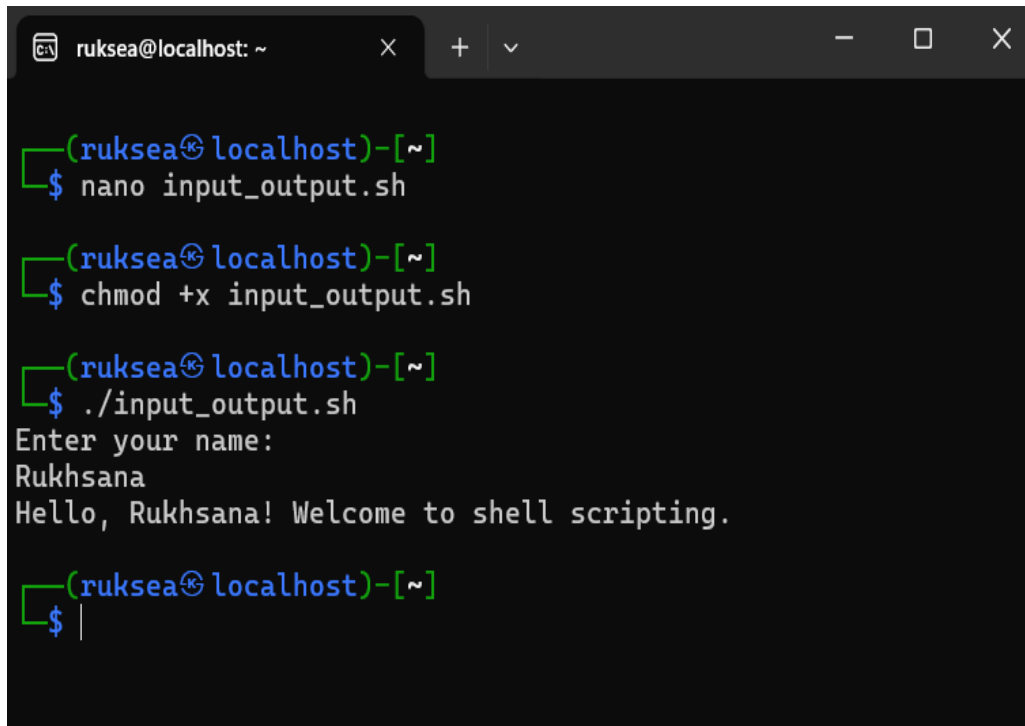
Terminal window showing the nano text editor editing a shell script named input_output.sh. The script contains comments and code for taking user input and displaying output. The terminal window title is 'ruksea@localhost: ~'. The nano editor status bar at the bottom shows various keyboard shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^F Where Is, ^\ Replace, ^K Cut, ^U Paste, ^T Execute, ^J Justify.

Save & Exit

- Press Ctrl + X → Y → Enter

Run using `chmod +x input_output.sh`

Output:

A terminal window with a dark background and light green text. The window title bar shows 'ruksea@localhost: ~' and standard window controls. The terminal shows a series of commands and their outputs. The user runs 'nano input_output.sh', then 'chmod +x input_output.sh', and finally './input_output.sh'. The script prompts for a name, which is 'Rukhsana', and then prints a welcome message. The prompt returns to the shell.

```
(ruksea@localhost)-[~]  
$ nano input_output.sh  
  
(ruksea@localhost)-[~]  
$ chmod +x input_output.sh  
  
(ruksea@localhost)-[~]  
$ ./input_output.sh  
Enter your name:  
Rukhsana  
Hello, Rukhsana! Welcome to shell scripting.  
  
(ruksea@localhost)-[~]  
$ |
```

Practical no.11. Shell Scripting - Data Types.

Shell scripting primarily deals with strings. There are no distinct data types like in traditional programming languages. However, we can simulate different types:

- **String**
- **Integer**
- **Array**
- **Boolean (emulated using logic)**

Bash does not enforce data types strictly, but operations can be done conditionally to check or simulate behavior.

OUTPUT:

```
GNU nano 8.2          datatype.sh *
#!/bin/bash

#string type
name="Rukhsana"
echo "String example: Name is $name"

#integer type
age = 25
echo "Integer Example: Age is $age"

#Arithmetic operation
sum=$((age-5))
echo "Age after 5 years: $sum"

#Array Type
fruits=("Apple","Banana")
echo "Array Example: First fruit is ${fruits[0]}"

#loop through array
echo "All fruits:"
for fruits in "${fruits[@]}"
do
echo "$fruits"
done
```

```
(kali@kali)-[~]
$ ./datatype.sh
String example: Name is Rukhsana
Integer Example: Age is 25
Age after 5 years: 5
Array Example: First fruit is Apple,Banana
All fruits:
Apple,Banana
```


Practical No. 12: Shell Script – Arithmetic Operations

Aim:

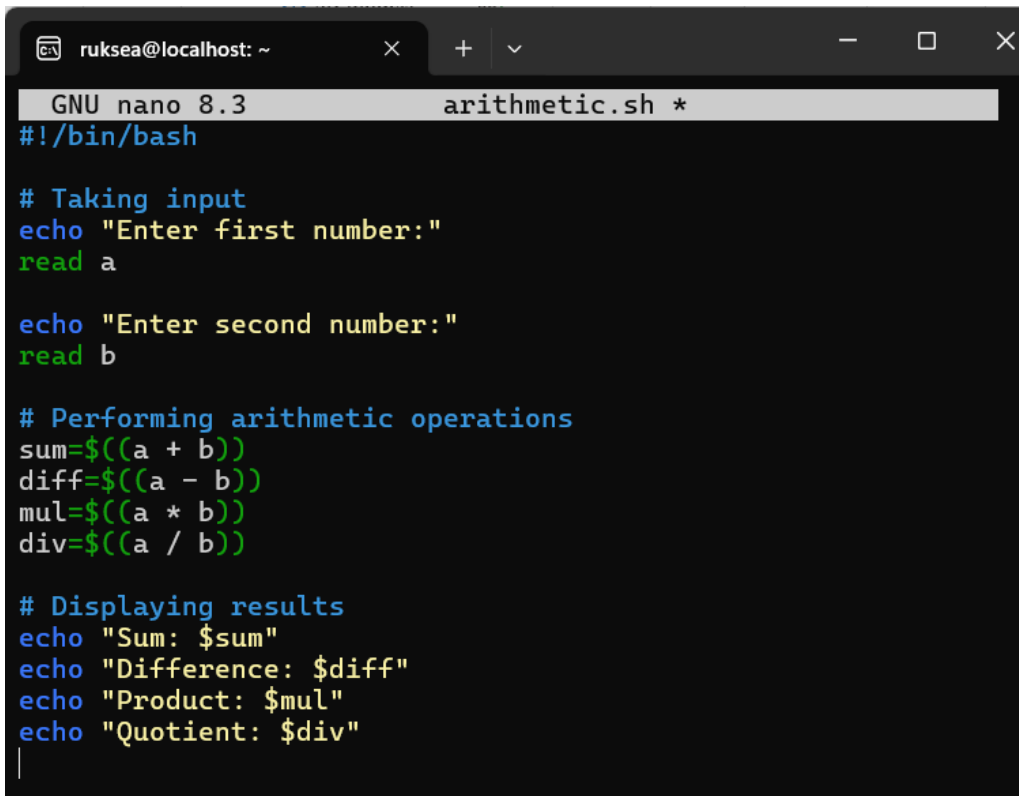
To perform basic arithmetic operations using shell script.

Requirements:

- Linux environment
- bash shell

Procedure:

1. Open terminal and type: nano arithmetic.sh
2. Write the script:



```
ruksea@localhost: ~  
GNU nano 8.3 arithmetic.sh *  
#!/bin/bash  
  
# Taking input  
echo "Enter first number:"  
read a  
  
echo "Enter second number:"  
read b  
  
# Performing arithmetic operations  
sum=$((a + b))  
diff=$((a - b))  
mul=$((a * b))  
div=$((a / b))  
  
# Displaying results  
echo "Sum: $sum"  
echo "Difference: $diff"  
echo "Product: $mul"  
echo "Quotient: $div"  
|
```

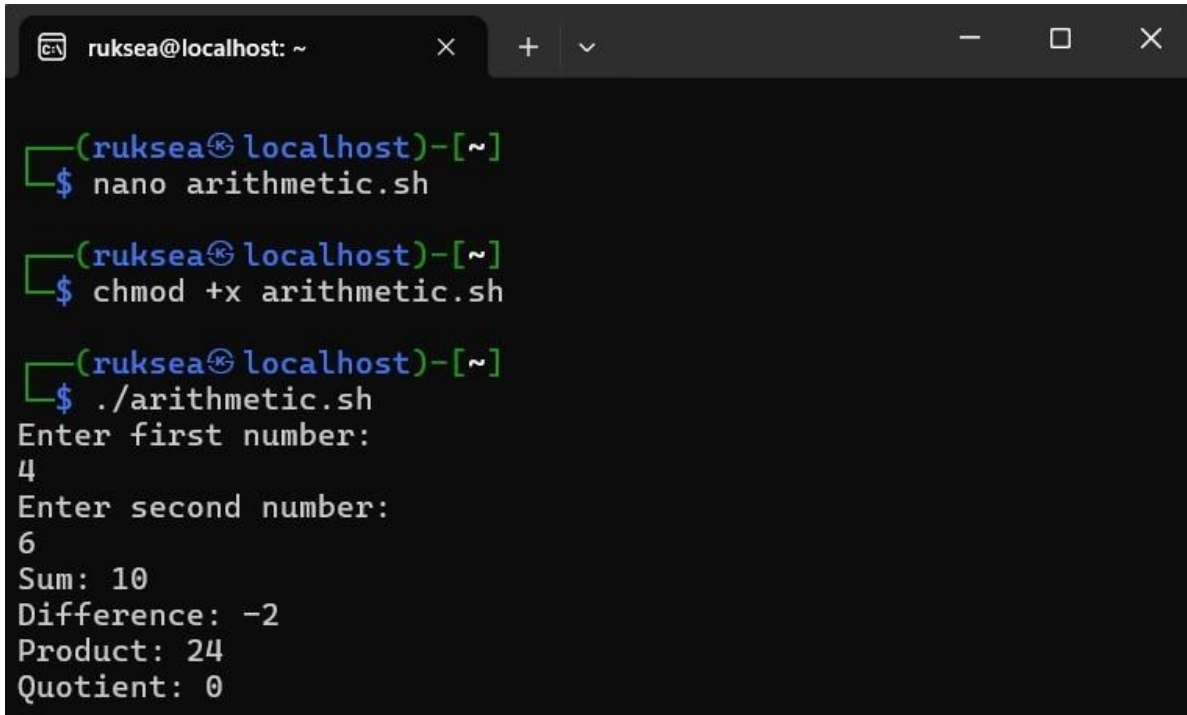
Save & Exit

- Press Ctrl + X → Y →

Enter Run using chmod +x

arithmetic.sh

Output:



```
ruksea@localhost: ~  
  
(ruksea@localhost)-[~]  
$ nano arithmetic.sh  
  
(ruksea@localhost)-[~]  
$ chmod +x arithmetic.sh  
  
(ruksea@localhost)-[~]  
$ ./arithmetic.sh  
Enter first number:  
4  
Enter second number:  
6  
Sum: 10  
Difference: -2  
Product: 24  
Quotient: 0
```

Practical No. 13: Shell Script – If Else Condition

Aim:

To write a shell script that uses if-else for decision making.

Requirements:

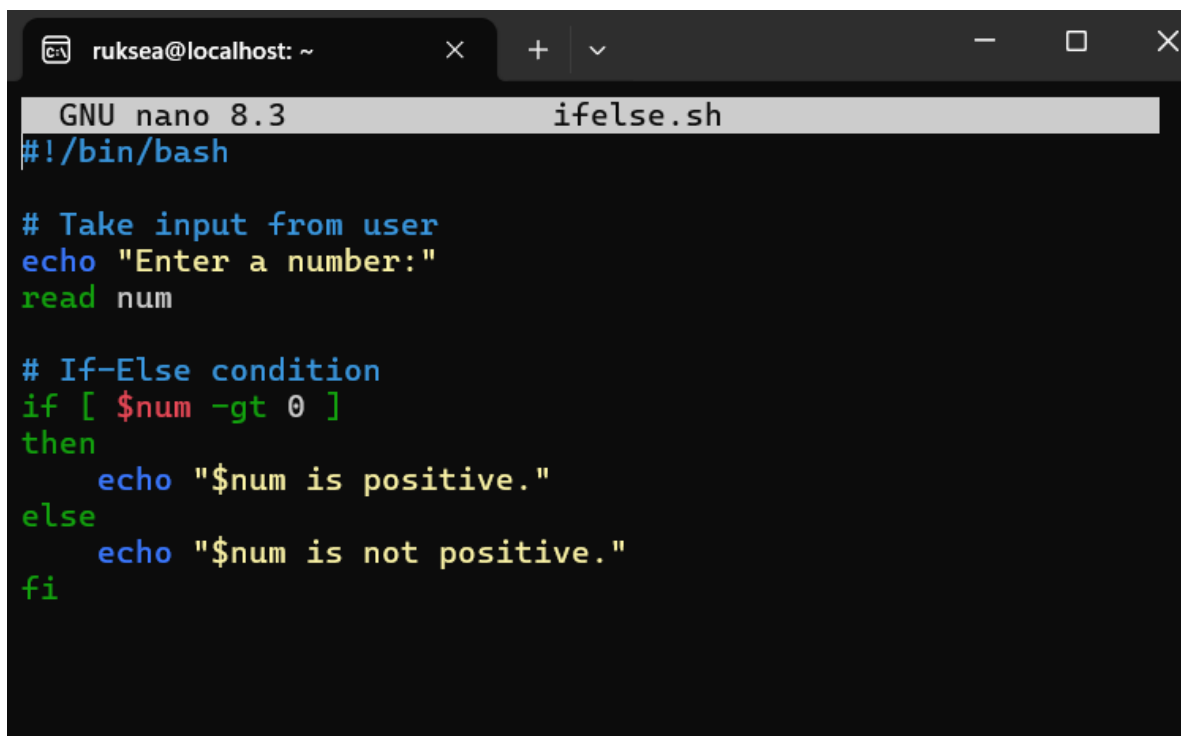
- bash shell

Procedure:

1. Open terminal and type: nano ifelse.sh
2. Write the script:

Syntax:

```
if [ condition ]  
then  
    # commands if condition is true  
else  
    # commands if condition is false  
fi
```



```
ruksea@localhost: ~  
GNU nano 8.3 ifelse.sh  
#!/bin/bash  
  
# Take input from user  
echo "Enter a number:"  
read num  
  
# If-Else condition  
if [ $num -gt 0 ]  
then  
    echo "$num is positive."  
else  
    echo "$num is not positive."  
fi
```

Save & Exit

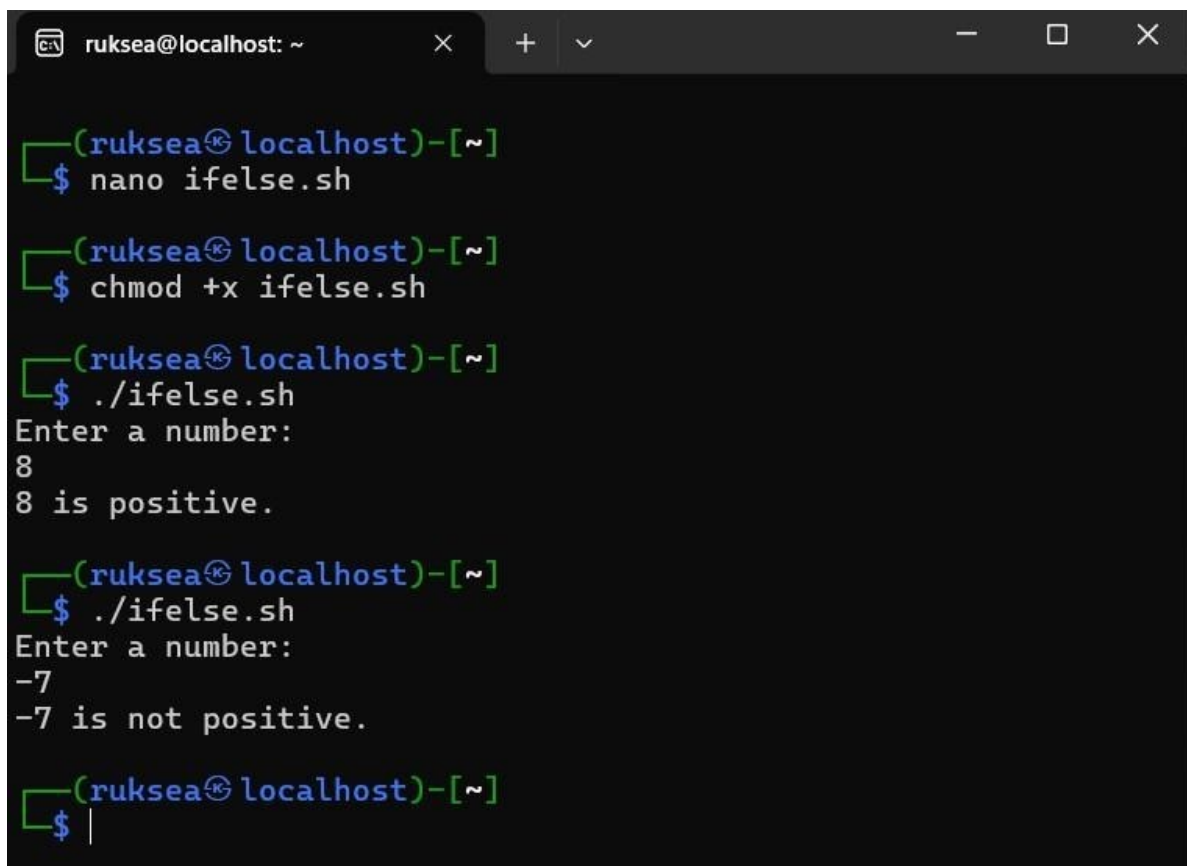
- Press Ctrl + X → Y →

Enter Run using chmod +x

ifelse.sh

Output:

Displays if the entered number is positive, negative, or zero.

A terminal window titled 'ruksea@localhost: ~' with standard window controls. The terminal shows a series of commands and their outputs. The user enters 'nano ifelse.sh' to create the script, then 'chmod +x ifelse.sh' to make it executable. They then run './ifelse.sh' twice. The first time, they enter '8' and the script outputs '8 is positive.'. The second time, they enter '-7' and the script outputs '-7 is not positive.'. The prompt is currently at the start of a new line.

```
(ruksea@localhost)-[~]
$ nano ifelse.sh

(ruksea@localhost)-[~]
$ chmod +x ifelse.sh

(ruksea@localhost)-[~]
$ ./ifelse.sh
Enter a number:
8
8 is positive.

(ruksea@localhost)-[~]
$ ./ifelse.sh
Enter a number:
-7
-7 is not positive.

(ruksea@localhost)-[~]
$ |
```

Practical no.14. Shell Scripting - While Control Statement

Aim

To understand and implement the while control statement in shell scripting to perform repetitive tasks.

Theory:

The while loop in shell scripting is used to execute a block of commands as long as the given condition is true. It is commonly used when the number of iterations is not known in advance.

Syntax:

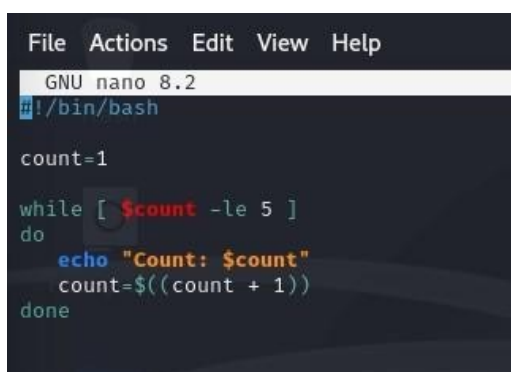
```
while [ condition ]
```

```
do
```

```
    # commands
```

```
done
```

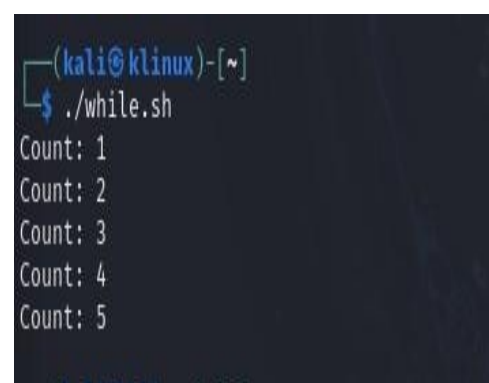
OUTPUT:



```
File Actions Edit View Help
GNU nano 8.2
#!/bin/bash

count=1

while [ $count -le 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done
```



```
(kali@linux)-[~]
$ ./while.sh
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Practical No. 15: Shell Script – Looping (For Loop)

Aim:

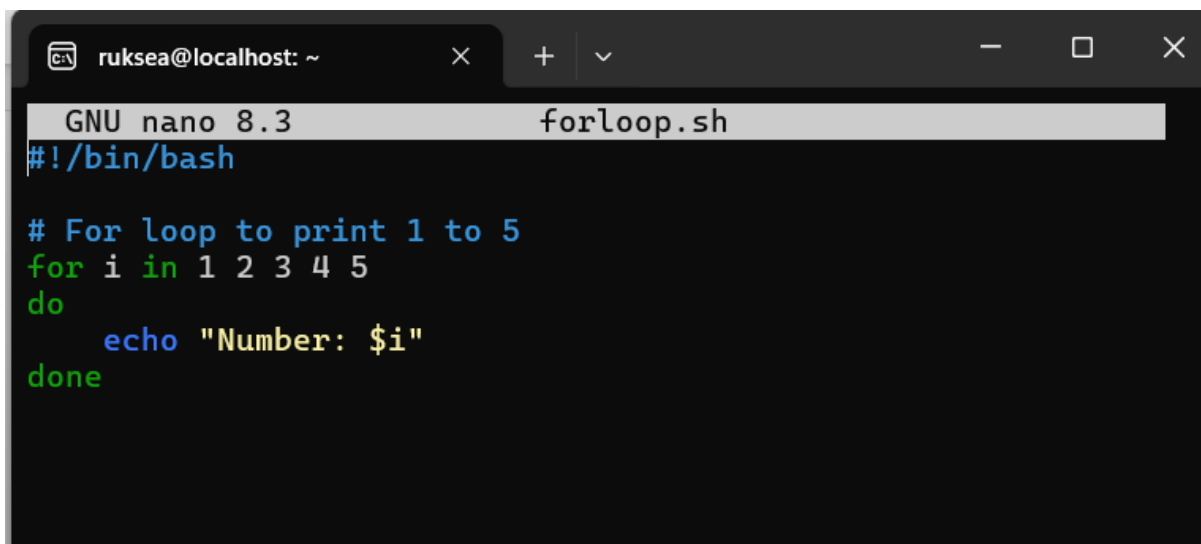
To write a shell script using a for loop to print numbers.

Requirements:

- bash shell

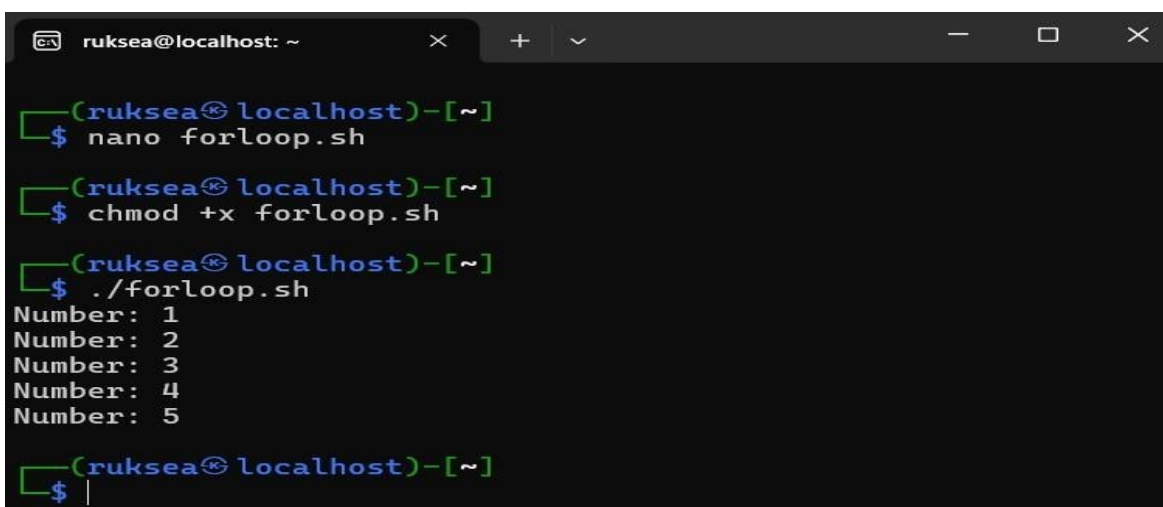
Procedure:

1. Open terminal and type: nano forloop.sh
2. Write the script:



```
ruksea@localhost: ~  
GNU nano 8.3 forloop.sh  
#!/bin/bash  
  
# For loop to print 1 to 5  
for i in 1 2 3 4 5  
do  
    echo "Number: $i"  
done
```

Output:



```
(ruksea@localhost)-[~]  
$ nano forloop.sh  
  
(ruksea@localhost)-[~]  
$ chmod +x forloop.sh  
  
(ruksea@localhost)-[~]  
$ ./forloop.sh  
Number: 1  
Number: 2  
Number: 3  
Number: 4  
Number: 5  
  
(ruksea@localhost)-[~]  
$ |
```